CA-3 · Agentic AI Proof of Concept

Scholarship Screening Desk

Six AI agents that screen scholarship applications, check them against the rules and explain every decision.



Student 1	**Saafalya Sengupta · PRN 24070126158**
Student 2	**Swastik Gavhane · PRN 24070126224**
Class / Division	**AIML C1**
Domain	**Education: scholarship administration**
Faculty	**Harshal Raut**
Date	**24 October 2026**

Run locally: double-click run_windows.bat (Ollama + Streamlit)

Code: https://github.com/Saafalya/scholardesk-

1. Problem statement and domain

Colleges run merit-cum-means scholarships every year. A student fills in a form with family income, previous-year CGPA, attendance and category, then attaches an income certificate, a marksheet and, for reserved categories, a category certificate. Office staff check each application by hand against the scheme rules and prepare a shortlist for the committee.

Problem statement. Manual screening of scholarship applications is slow, inconsistent between reviewers, and poor at catching mismatches between what a student writes on the form and what the marksheet actually says. Rejected students rarely get a clear reason.

Why this domain

Education fits an agentic approach well for three reasons. The rules are explicit, so every decision can be checked. The inputs are messy free text, which is where a language model helps. And the cost of a wrong call is real: a deserving student misses support, or money goes to someone who already holds a full award. That makes validation and explanation as important as speed, which is exactly what an agent team with a dedicated Critic can show.

2. Existing challenge and proposed solution

How it works today

Step	What staff do now	What goes wrong
Read the form	Type income, CGPA and attendance into a sheet by hand	Typing errors, missed fields
Check documents	Open each attachment and tick a box	Missing papers noticed late, after the shortlist
Apply the rules	Compare against the income limit and the category cutoffs	Different reviewers read rules differently
Cross-check	Compare form CGPA with the marksheet, if time allows	Inflated CGPA often slips through
Check other awards	Search the registry for full awards	Double funding when the search is skipped
Shortlist	Sort by CGPA and income in a spreadsheet	No written reason for each rejection

Proposed solution

Scholarship Screening Desk hands this routine work to six cooperating agents. A Planner decides which checks apply. An Extractor uses a local language model (Ollama) to pull fields out of the free-text form and marksheet. A Critic independently re-reads the source text, checks that every number the model returned actually appears in it, and sends the case back when a field is wrong. A Validator runs every scheme rule as a Python tool. A Decision agent turns the rule results into one of three verdicts and scores eligible students. A Reporter writes a plain-English note for the reviewer. In batch mode the system ranks eligible students and fills the number of awards available.

The human reviewer stays in charge: the system recommends and explains, and every step is shown in an agent trace.

3. Agent roles, responsibilities and orchestration

Agent	Type	Responsibility	Uses the model?
Planner	Planner	Reads the case and lists the checks to run, with notes such as relaxed cutoffs for reserved categories. A guardrail adds back any mandatory check the model leaves out.	Yes
Extractor	Research	Turns the free-text form and marksheet into structured fields: name, category, income, form CGPA, marksheet CGPA, attendance and other awards.	Yes
Critic	Critic	Runs two checks on the extraction: a separate pattern reader compares field by field, and a grounding check confirms every number appears in the source text (5.4 lakh counts as 540000). On a mismatch it sends the case back with its notes; if retries run out, it replaces or clears the field. After the verdict it checks the verdict agrees with every rule result.	No
Validator	Executor	Calls six Python tools: check_income, check_cgpa, check_attendance, check_documents, check_consistency and check_registry. Each returns pass, fail, missing or unclear with a reason.	No
Decision	Business automation	Any failed rule means not eligible. Any missing or unclear item means needs documents. Otherwise eligible, with a merit-cum-means score.	No
Reporter	Business automation	Writes a two to three sentence note naming the exact rule that failed or the document that is missing.	Yes

Orchestration workflow

A Python orchestrator runs the agents in a fixed order and passes one shared case state between them. The state holds the plan, the extracted fields, the rule results, the verdict, the note and a trace of every step. The flow for one application is:

Planner → Extractor → Critic (verify fields) → [send back to Extractor, up to N times] → Validator → Decision → Critic (verify verdict) → Reporter.

Three decision points make this agentic rather than a fixed script. The Planner chooses the checks per case. The Critic decides whether to accept the extraction, send it back, or override it. The Decision agent chooses between three outcomes, and in batch mode the orchestrator decides who is awarded and who is waitlisted based on the number of slots. The number of send-backs is a setting in the sidebar.

Merit-cum-means score = 100 × (0.6 × CGPA ÷ 10 + 0.4 × (1 − income ÷ income limit)). It rewards strong marks and greater financial need together, so a student with a 7.8 CGPA and low income can rank above a student with 9.1 and a higher income.

4. Architecture

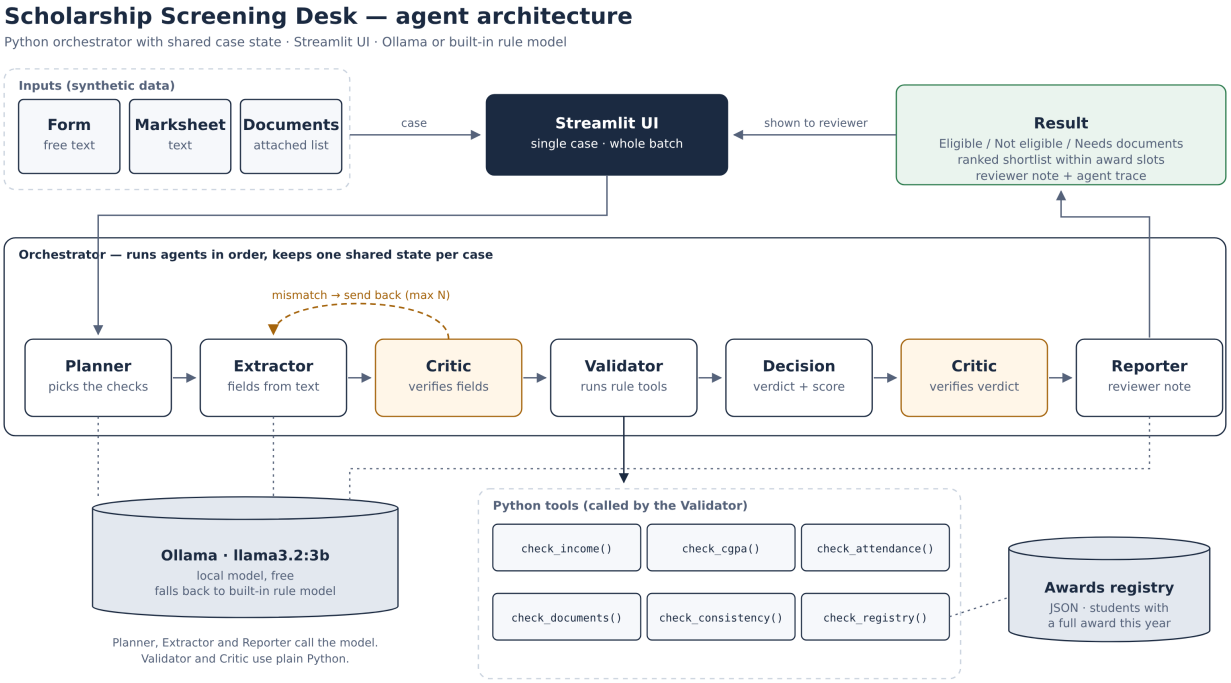


Figure 1. Agents, data flow, tools and decision points. Amber boxes are the two Critic checkpoints; the dashed amber arrow is the send-back loop.

Layer	What it is	Technology
Interface	Single-case screening, batch shortlist, rules and design tabs	Streamlit
Orchestration	Runs agents in order, holds shared case state, controls retries	Plain Python classes
Model	Planner, Extractor and Reporter prompts; JSON-schema structured output	Ollama, llama3.2:3b (local, free)
Fallback model	Rule-based model with the same interface when Ollama is not reachable	Python regex
Tools	Six rule functions the Validator calls	Python
Data	12 synthetic applications and a synthetic awards registry	JSON files
Running it	One-click Windows launcher on the laptop; optional backup link	run_windows.bat; Render (rule model)

5. Working Streamlit PoC

The app runs on the laptop with Ollama. Double-clicking run_windows.bat sets up Python packages, starts Ollama, pulls llama3.2:3b if it is missing and opens the app. The sidebar shows whether Ollama is running, which models are installed, and a Test the model button that loads the model before the demo. While a case runs, a strip of six agent tiles lights up as each agent works. Every step in the trace is labelled with who did the work (Ollama, the rule model or plain Python) and how long it took, and the exact prompt and reply for each model call can be opened.

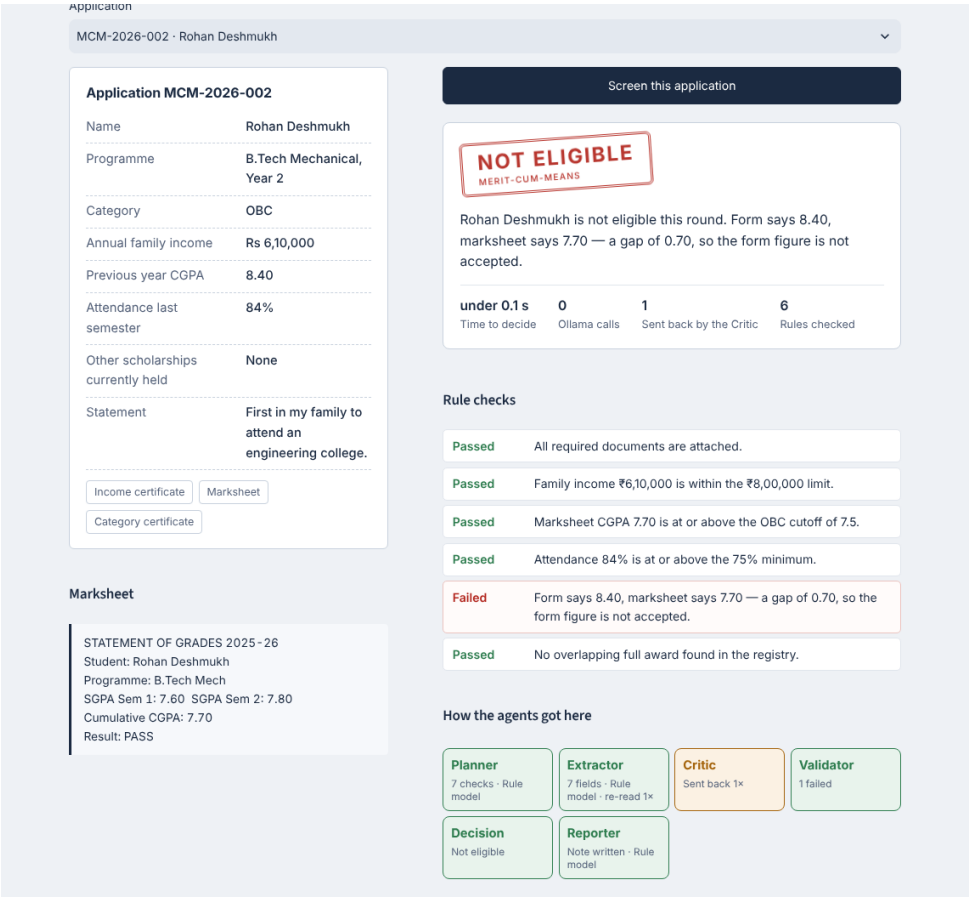


Figure 2. Screening one application. The verdict card shows the stamp, the reviewer note, time taken, model calls and send-backs. Here the demo switch made the Extractor slip, the Critic sent the case back (amber tile), and the case was rejected for an inflated CGPA.

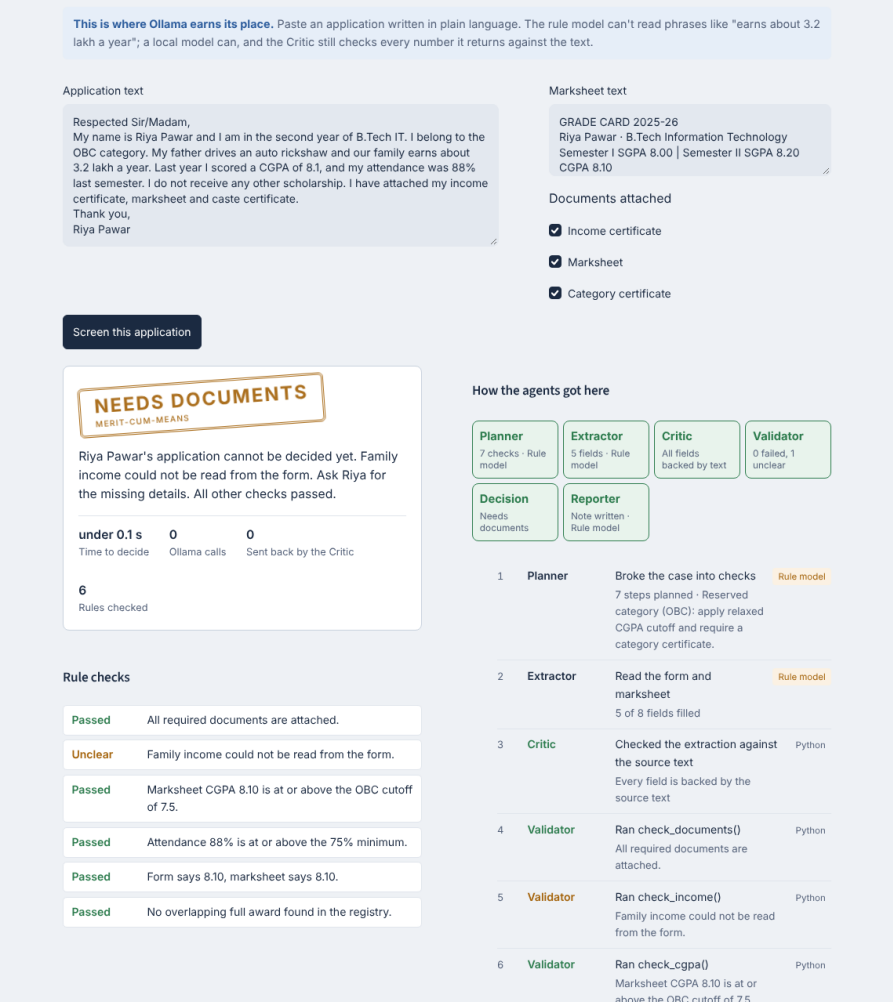


Figure 3. The Paste a new application tab, run here with the built-in rule model. It cannot read “earns about 3.2 lakh a year”, so income is marked unclear and the case needs details. With Ollama, the Extractor reads it as ₹3,20,000 and the Critic's grounding check accepts it because 3.2 lakh appears in the text; the automated tests confirm this path with simulated Ollama replies.

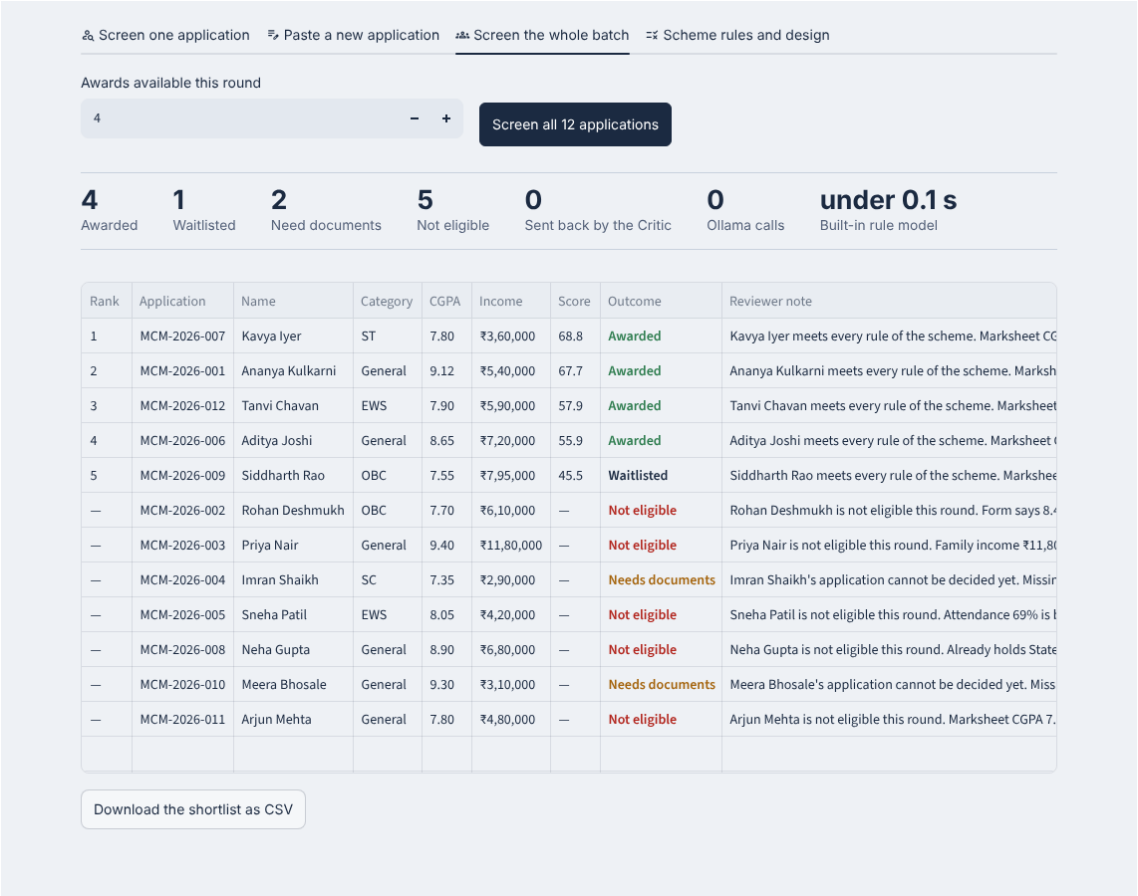


Figure 4. Batch screening of all 12 applications with four awards available.

Synthetic data

The 12 applications are invented but realistic, and each one is built to test a specific rule: a clean pass, an inflated CGPA, income above the limit, a missing category certificate, low attendance, a student who already holds a full award, a missing income certificate, a borderline case just inside the limits, and reserved-category students who pass on the relaxed cutoff.

6. Output validation and results

Validation happens at two levels: inside the system, where the Critic checks the other agents at runtime, and outside it, where automated tests compare results against hand-checked answers.

Results on the 12 synthetic applications

Application	Name	Category	Verdict	Score	Outcome
MCM-2026-007	Kavya Iyer	ST	Eligible	68.8	Awarded
MCM-2026-001	Ananya Kulkarni	General	Eligible	67.7	Awarded
MCM-2026-012	Tanvi Chavan	EWS	Eligible	57.9	Awarded
MCM-2026-006	Aditya Joshi	General	Eligible	55.9	Awarded
MCM-2026-009	Siddharth Rao	OBC	Eligible	45.5	Waitlisted
MCM-2026-002	Rohan Deshmukh	OBC	Not eligible	—	Not eligible
MCM-2026-003	Priya Nair	General	Not eligible	—	Not eligible
MCM-2026-004	Imran Shaikh	SC	Needs documents	—	Needs documents
MCM-2026-005	Sneha Patil	EWS	Not eligible	—	Not eligible
MCM-2026-008	Neha Gupta	General	Not eligible	—	Not eligible
MCM-2026-010	Meera Bhosale	General	Needs documents	—	Needs documents
MCM-2026-011	Arjun Mehta	General	Not eligible	—	Not eligible

Validation checks

Check	How it is tested	Result
Verdict accuracy	Each of the 12 verdicts compared with a hand-checked label	12 of 12 correct
Critic catches extraction errors	Demo switch injects one realistic mistake per case (dropped digit, copied value, misread number)	12 of 12 caught and sent back
Verdicts survive injected errors	Same 12 cases with mistakes injected and one retry allowed	12 of 12 verdicts unchanged
Critic override with no retries	Mistakes injected and retries set to zero	12 of 12 verdicts unchanged
Planner guardrail	Simulated Ollama plan that lists only one check	All six checks restored
Real model path	Simulated Ollama reply with a wrong income, then a correct one	Caught; correct verdict
Plain-language input	Free-text application with income written as 3.2 lakh	Rule model: needs details; Ollama: eligible
Invented number	Simulated Ollama reply with an income that is not in the text	Grounding check sends it back
Decision consistency	Critic compares each verdict with the rule results	0 contradictions

All tests live in tests/test_pipeline.py and run with one command. In the batch run with four awards, Kavya Iyer (ST, CGPA 7.80, income ₹3,60,000) ranks first ahead of Ananya Kulkarni (General, CGPA 9.12, income ₹5,40,000), which shows the score weighing need alongside merit as the scheme intends.

Speed: with the built-in model the agents screen all 12 applications in under a second. With Ollama, each case needs three model calls (plan, extract, report), so the time per case depends on the laptop running the model. Either way, every decision comes with a written reason, which manual screening rarely provides.

7. Limitations

- **Synthetic data only.** The forms are clean text. Real applications arrive as scanned PDFs and photos, which would need OCR before the Extractor.
- **Pattern check shares code with the fallback model.** In rule-model mode the Critic's pattern check reads the text the same way as the fallback Extractor, so only the grounding check is independent. The Critic is most useful when Ollama does the extraction.
- **Small local model.** llama3.2:3b can misread numbers or return broken JSON. The Critic and the fallback catch this, but extraction is slower than a larger hosted model.
- **Runs on one laptop.** The full Ollama version needs the laptop running it. The optional Render link has too little memory for Ollama, so it uses the built-in rule model.
- **Rules are fixed in code.** A new scheme needs a developer to change rules.py.
- **No document authenticity check.** The system checks that documents are attached, not that they are genuine.

8. Future scope

- Add OCR so the Extractor can read scanned certificates and marksheets directly.
- Let the scholarship office edit rules in the app, so one system can run several schemes.
- Move orchestration to LangGraph and expose the rule tools over MCP, so other college systems can call them.
- Verify income certificates against government records such as DigiLocker, with the student's consent.
- Draft the email to each student with their result and, if needed, the documents to upload.
- Track reviewer overrides and use them to improve prompts and spot rules that need clarifying.

Appendix A. Scheme rules

Scheme: Merit-cum-Means Scholarship 2026-27 (synthetic). Family income at most ₹8,00,000 a year. Attendance at least 75%. Form CGPA must match the marksheet within 0.05. No full-tuition award already held this year.

Category	Minimum CGPA	Documents required
General	8.0	income certificate, marksheet
EWS	7.5	income certificate, marksheet, category certificate
OBC	7.5	income certificate, marksheet, category certificate
SC	7.0	income certificate, marksheet, category certificate
ST	7.0	income certificate, marksheet, category certificate

Appendix B. How to run

On Windows, double-click run_windows.bat. By hand on any system:

```
pip install -r requirements.txt

ollama pull llama3.2:3b

streamlit run app.py

python tests/test_pipeline.py
```

Code: https://github.com/Saafalya/scholardesk-
Backup link (rule model only): https://scholardesk-wpdp.onrender.com